Array-Based Max-Heap Task Scheduler

CS314 · Analysis of Algorithms

1. Problem

A task scheduler selects the most urgent waiting task. Each task is represented by a name, a numeric priority, and an arrival time. The program inserts 20 generated tasks into an array-based max-heap, repeatedly applies EXTRACT-MAX, and compares the result with a sorted-list implementation that re-sorts after each insertion.

2. Method

The heap is a complete binary tree stored in an array. With zero-based indexing, the parent of index i is (i – 1) // 2, and the children are 2i + 1 and 2i + 2. The max-heap property keeps each parent at least as urgent as its children, so the root contains the next task to run. Priority is the primary key; equal priorities are ordered by earlier arrival time, then by task name for a deterministic result.

INSERT appends a task at the end of the array and sifts it up while it outranks its parent. EXTRACT-MAX removes the root, moves the last element to the root, reduces the heap size, and calls MAX-HEAPIFY to sift the element down. This follows the lecture's priority queue operations.

Operation / approach	Cost
MAXIMUM (read root)	O(1)
INSERT	O(log n)
EXTRACT-MAX	O(log n)
Sorted list: re-sort each insertion	O(n log n) per insertion

3. Complexity and comparison

For n tasks, inserting all tasks into the heap takes O(n log n), and extracting all tasks takes O(n log n); the heap requires O(n) storage. Each individual INSERT and EXTRACT-MAX is O(log n), as specified in the assignment. The sorted-list version sorts the current list after each insertion, costing O(k log k) at insertion k and $O(n^2 \log n)$ overall. Once sorted in descending order, extracting the first element uses O(n) total shifting work; this does not change the dominant insertion cost. Re-sorting can be simple for small inputs, while the heap maintains priority order with logarithmic updates.

4. Experiment and conclusion

The program uses a fixed random seed so the same 20 priorities and arrival times are generated on each run. Both implementations receive the identical tasks and use the same tie-breaking rule; the printed run orders can therefore be compared directly. The expected result is that both approaches produce the same sequence, while their insertion costs differ as the queue grows. The max-heap is the appropriate structure for repeated scheduling because it preserves the maximum at the root and supports each priority queue update in O(log n).